

Part II: Angular File Anatomy

Table of Contents

- [Chapter 1: Anatomy of an Angular Component](#)
- [Chapter 2: Anatomy of an Angular Service](#)
- [Chapter 3: Anatomy of an Angular Guard](#)
- [Chapter 4: Anatomy of an Angular Interceptor](#)
- [Chapter 5: Anatomy of Angular Models and Interfaces](#)
- [Chapter 6: Anatomy of Angular Configuration Files](#)
- [Chapter 7: Angular Project Structure and Folder Organization](#)
- [Chapter 8: Angular Coding Conventions and Naming Standards](#)

Chapter 1: Anatomy of an Angular Component

"Every page, widget, or reusable UI element in Angular is built as a Component. While components may differ in functionality, they all follow the same overall structure."

Chapter Overview

In this chapter, we'll learn:

- The structure of an Angular Component
- Purpose of each section
- What belongs in each section
- How the sections work together
- Best practices for organizing components

We'll use `LoginComponent` as the reference example.

Standard Component Structure

A typical Angular component is organized like this:

```
// Imports

@Component({
  ...
})

export class LoginComponent {
```

```
// Properties

constructor(...) {}

// Methods

}
```

Every component in your project follows this pattern.

1. Imports

The top of the file contains all required imports.

Example

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../../core/services/auth.service';
```

Imports allow this file to use classes, interfaces, services, modules, and decorators defined elsewhere.

Typical imports include:

- Angular Core
- Angular Material
- Forms
- Router
- RxJS
- Models
- Services

2. @Component Decorator

Immediately after the imports comes the component metadata.

```
@Component({
  selector: 'app-login',
  standalone: true,
  imports: [...],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
```

The decorator tells Angular how this component should behave.

It is configuration, not business logic.

3. Component Metadata

The most common metadata properties are:

Property	Purpose
selector	HTML tag for the component
standalone	Marks the component as standalone
imports	Modules/components used by this component
templateUrl	HTML template
styleUrl	Component stylesheet

Most components use these same properties.

4. Component Class

The actual component begins here.

```
export class LoginComponent {  
  
}
```

This class contains all the logic for the UI.

5. Properties

Properties represent the component's state.

Example

```
loading = false;  
  
errorMessage = '';  
  
hidePassword = true;
```

```
loginForm!: FormGroup;
```

These values are read by the HTML template.

A good practice is to declare all properties near the top of the class.

6. Constructor

The constructor is mainly used for Dependency Injection.

Example

```
constructor(  
    private authService: AuthService,  
    private router: Router  
) {}
```

Avoid putting business logic inside constructors.

Its primary responsibility is receiving dependencies.

7. Methods

Methods contain the component's behavior.

Example

```
login()  
  
logout()  
  
onGenerate()  
  
pollJobStatus()
```

Methods usually:

- respond to button clicks
- call services
- update properties

- navigate
- process user actions

8. External Files

Each component usually has three files.

```
login.component.ts  
  
↓  
  
Logic  
  
-----  
  
login.component.html  
  
↓  
  
UI  
  
-----  
  
login.component.css  
  
↓  
  
Styling
```

Keeping them separate improves readability and maintenance.

9. Typical Component Layout

A well-organized component usually follows this order.

```
Imports  
  
↓  
  
@Component  
  
↓  
  
Properties  
  
↓
```

Constructor

↓

Public Methods

↓

Private Methods (if any)

Keeping the same order across the project makes code easier to navigate.

Best Practices

- ✓ Keep one component responsible for one feature.
- ✓ Keep constructors small.
- ✓ Group properties together.
- ✓ Place public methods before private helper methods.
- ✓ Keep business logic inside services whenever possible.
- ✓ Keep templates focused on presentation.
- ✓ Use consistent ordering in every component.

Interview Questions

1. What are the main sections of an Angular component?
2. What is the purpose of the `@Component` decorator?
3. What belongs inside a constructor?
4. What are component properties?
5. What are component methods?
6. Why are HTML and CSS kept in separate files?
7. How should a component be organized for readability?

Chapter 2: Anatomy of an Angular Service

"Components handle the UI. Services handle the business logic. This separation keeps applications modular, reusable, and easy to maintain."

Chapter Overview

In this chapter, we'll learn:

- The structure of an Angular Service
- Purpose of each section
- What belongs inside a Service
- Singleton Services
- How Components use Services
- Best practices

We'll use `AuthService`, `AuthService`, `TokenService`, and `AuthStateService` as examples.

Standard Service Structure

A typical Angular service is organized like this.

```
// Imports

@Injectable({
  providedIn: 'root'
})

export class AuthService {

  // Fields

  constructor(...) {}

  // Public Methods

  // Private Methods (optional)

}
```

Unlike Components, Services do not have HTML or CSS files.

1. Imports

The file begins with the required imports.

Example

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
```

A service commonly imports:

- Angular classes
- HttpClient
- RxJS
- Models
- Other services

2. @Injectable Decorator

Every service is decorated with

```
@Injectable({
  providedIn: 'root'
})
```

This tells Angular that the class participates in Dependency Injection.

Without `@Injectable`, Angular cannot inject dependencies into the service.

3. providedIn: 'root'

```
providedIn: 'root'
```

This registers the service globally.

Only one instance of the service is created for the entire application.

This is called a **Singleton Service**.

Entire Application

|



One AuthService Instance

4. Service Class

The service begins here.

```
export class AuthService {  
  
}
```

This class contains reusable business logic.

Unlike Components, Services never contain UI logic.

5. Fields

Services usually store configuration values.

Example

```
private apiUrl =  
'http://localhost:5260/api/auth';
```

Fields commonly store:

- API URLs
- Configuration values
- Private helper objects

6. Constructor

Services receive dependencies through the constructor.

Example

```
constructor(  
  
  private http: HttpClient,
```

```
private tokenService: TokenService  
  
) {}
```

Typical dependencies include:

- HttpClient
- Router
- Other Services

7. Public Methods

Most service functionality is exposed through public methods.

Examples from your project

```
login()  
  
generate()  
  
getStatus()  
  
saveToken()  
  
getToken()  
  
logout()
```

Components call these methods whenever they need functionality.

8. Private Methods

Some services may contain helper methods.

Example

```
private createHeaders() {  
  
}
```

Private methods are only used internally by the service.

Your current services are simple enough that they don't require private methods yet.

9. Service Responsibilities

Each service should have one clear responsibility.

Your project already follows this pattern.

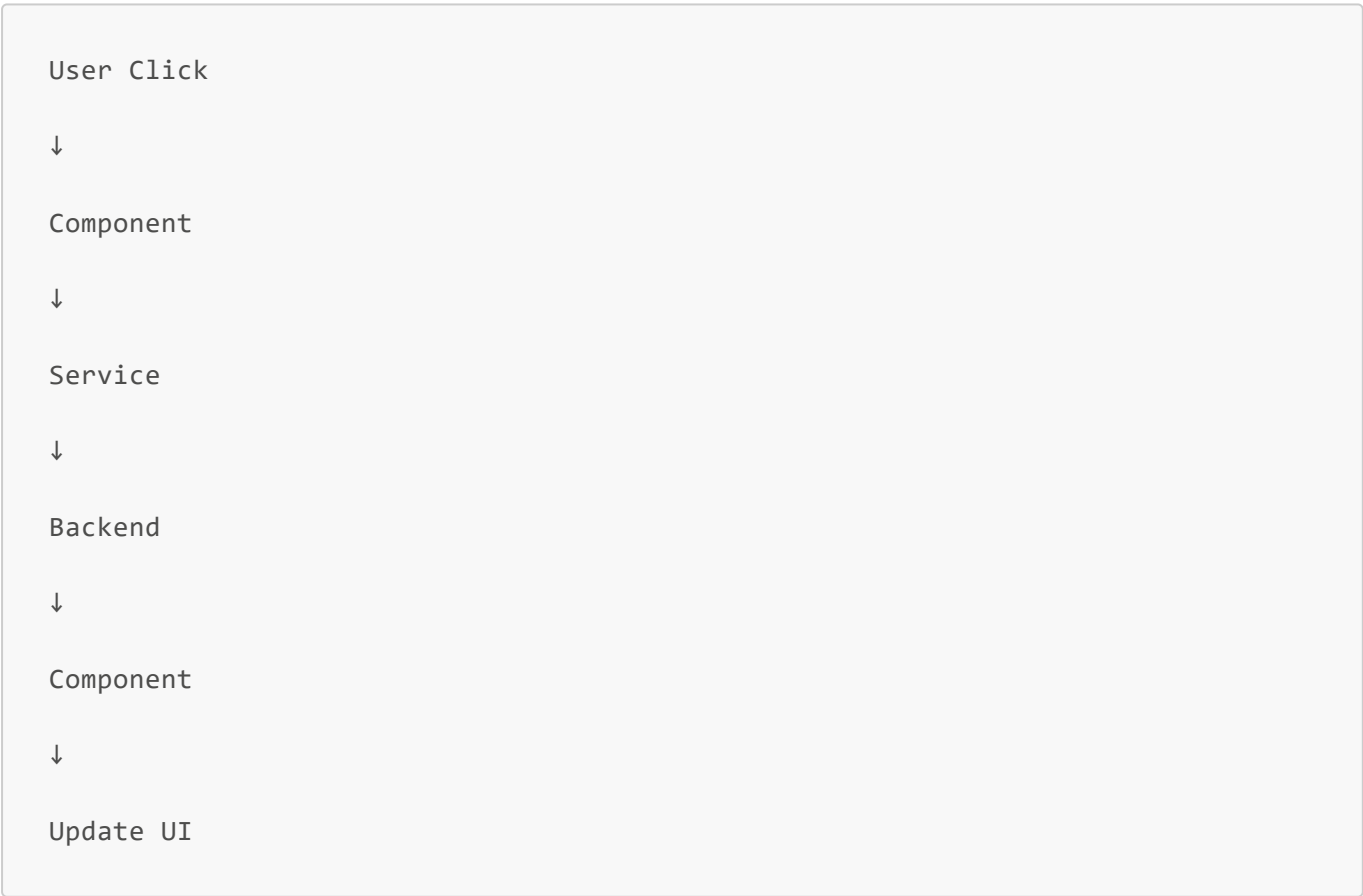
Service	Responsibility
AuthService	Authentication API calls
AiService	AI API calls
TokenService	JWT storage and retrieval
AuthStateService	Authentication state (logout, login status)

Keeping responsibilities separate makes the code easier to maintain.

10. Component → Service Relationship

Components should never contain business logic.

Instead they delegate work to services.



The component manages the UI.

The service performs the work.

11. Service Relationships in Your Project

Your current services work together like this.

LoginComponent

|



AuthService

|



TokenService

and

MainLayoutComponent

|



AuthStateService

|



TokenService

Each service has a focused responsibility.

12. Typical Service Layout

A well-organized service usually follows this order.

Imports

↓

@Injectable

↓

Fields

↓

Constructor

↓

Public Methods

↓

Private Methods

Using the same structure throughout the project improves readability.

Best Practices

- ✓ One service should have one responsibility.
 - ✓ Keep UI logic inside components.
 - ✓ Keep business logic inside services.
 - ✓ Keep fields private whenever possible.
 - ✓ Use Dependency Injection instead of creating objects manually.
 - ✓ Reuse services instead of duplicating logic.
 - ✓ Prefer small, focused services over large "God Services."
-

Interview Questions

1. What is an Angular Service?
2. Why do we use Services?
3. What does @Injectable do?
4. What is providedIn: 'root'?
5. What is a Singleton Service?

6. What belongs inside a Service?
7. Why shouldn't Components contain business logic?
8. Explain the responsibilities of the services in your Smart AI Dashboard.
9. How do Components communicate with Services?
10. How should a Service be organized for readability?

Chapter 3: Anatomy of an Angular Guard

"Guards control navigation. They decide whether a user is allowed to access a route before Angular loads the requested page."

Chapter Overview

In this chapter, we'll learn:

- The structure of an Angular Guard
- Purpose of each section
- Functional Guards
- Route Protection
- How Guards fit into Routing
- Best practices

We'll use your `auth.guard.ts` as the reference example.

Standard Guard Structure

A typical functional guard is organized like this.

```
// Imports

export const authGuard: CanActivateFn = () => {

  // Dependencies

  // Logic

  // Return Result

};
```

Unlike Components and Services, Guards are functions rather than classes.

1. Imports

The file begins with the required imports.

Example

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';

import { TokenService } from '../services/token.service';
```

Common imports include:

- Angular Router
- inject()
- Services required by the guard

2. Guard Declaration

A functional guard is declared like this.

```
export const authGuard:
CanActivateFn = () => {

}
```

`CanActivateFn` tells Angular that this function decides whether a route can be activated.

3. Dependency Injection

Since functional guards do not have constructors, Angular provides the `inject()` function.

Example

```
const tokenService =
inject(TokenService);

const router =
inject(Router);
```

This is equivalent to constructor injection inside Components and Services.

4. Guard Logic

The guard performs the required checks.

Your project checks whether a JWT exists.

```
if (tokenService.isLoggedIn()) {  
    return true;  
}
```

The logic should remain simple and focused.

5. Navigation

If the check fails, the guard redirects the user.

```
router.navigate(['/login']);
```

This sends unauthenticated users to the Login page.

6. Return Value

Every guard must return a result.

Typical return values are:

```
true
```

Allow navigation.

```
false
```

Block navigation.

Your guard currently returns one of these two values.

7. Route Relationship

Guards are attached inside routing.

Example

```
{  
  path: '',  
  component: MainLayoutComponent,  
  canActivate: [authGuard]  
}
```

Whenever this route is accessed,

Angular automatically executes the guard first.

8. Current Authentication Flow

Your guard currently performs this sequence.

User Requests Route

↓

authGuard

↓

Token Exists?

↓

Yes

↓

Load Component

No

↓

```
Navigate to Login
```

9. Current Responsibilities

Your `authGuard` currently handles:

- Checking login status
- Blocking unauthorized access
- Redirecting to Login

Nothing else.

Keeping guards focused makes them easier to maintain.

10. Future Guards

Your project will soon introduce another guard.

```
GuestGuard
```

Its responsibility is the opposite.

```
Already Logged In?
```

```
↓
```

```
Yes
```

```
↓
```

```
Dashboard
```

```
-----
```

```
No
```

```
↓
```

```
Allow Login Page
```

Together, these two guards will protect both authenticated and guest-only routes.

11. Typical Guard Layout

A well-organized guard usually follows this order.

Imports

↓

Guard Declaration

↓

Dependency Injection

↓

Validation Logic

↓

Navigation (if needed)

↓

Return Result

Best Practices

- ✓ Keep guards small.
 - ✓ One guard should have one responsibility.
 - ✓ Avoid business logic inside guards.
 - ✓ Use services to perform authentication checks.
 - ✓ Keep redirection simple.
 - ✓ Reuse guards across multiple routes.
-

Interview Questions

1. What is an Angular Guard?
2. Why do we use Guards?

3. What is `CanActivateFn`?
4. Why does a functional guard use `inject()` instead of a constructor?
5. What values can a guard return?
6. Where are guards configured?
7. Explain how your `authGuard` works.
8. What is the purpose of a `GuestGuard`?
9. What responsibilities should a guard have?
10. How should a guard be organized for readability?

Chapter 4: Anatomy of an Angular Interceptor

"Interceptors sit between your application and the backend. Every HTTP request and response passes through them, making them the ideal place for cross-cutting concerns such as authentication, logging, error handling, caching, and request modification."

Chapter Overview

In this chapter, we'll learn:

- The structure of an Angular Interceptor
- Purpose of each section
- Functional Interceptors
- Request Interception
- Response Interception
- How Interceptors fit into the HTTP pipeline
- Best practices

We'll use your `auth.interceptor.ts` as the reference example.

Standard Interceptor Structure

A typical functional interceptor is organized like this.

```
// Imports

export const authInterceptor: HttpInterceptorFn = (
  req,
```

```
    next

) => {

    // Dependencies

    // Logic

    // Forward Request

};
```

Like Guards, modern Angular Interceptors are functions rather than classes.

1. Imports

The file begins with the required imports.

Example

```
import { inject } from '@angular/core';

import { HttpInterceptorFn }
from '@angular/common/http';

import { TokenService }
from '../services/token.service';
```

Common imports include:

- HttpInterceptorFn
 - inject()
 - Required services
-

2. Interceptor Declaration

Every functional interceptor is declared like this.

```
export const authInterceptor:

HttpInterceptorFn = (

    req,

    next
```

```
) => {  
  
}
```

`HttpInterceptorFn` tells Angular that this function intercepts every HTTP request.

3. Parameters

Every interceptor receives two parameters.

```
req  
  
next
```

`req`

Represents the outgoing HTTP request.

Contains:

- URL
 - Method
 - Headers
 - Body
 - Query Parameters
-

`next`

Represents the next step in the HTTP pipeline.

Calling

```
next(req)
```

passes the request to the next interceptor or ultimately to the backend.

4. Dependency Injection

Like Guards, functional Interceptors use `inject()`.

Example

```
const tokenService =  
inject(TokenService);
```

This allows the interceptor to access application services.

5. Interceptor Logic

Your interceptor performs one simple responsibility.

```
const token =  
tokenService.getToken();
```

It retrieves the stored JWT.

The logic should remain focused and lightweight.

6. Request Modification

If a token exists,

your interceptor creates a modified copy.

```
const authRequest =  
req.clone({  
  
  setHeaders: {  
  
    Authorization:  
      `Bearer ${token}`  
  
  }  
  
});
```

The cloned request contains the Authorization header.

7. Forwarding the Request

Once the request is ready,

the interceptor forwards it.

```
return next(authRequest);
```

If no token exists,

the original request is forwarded instead.

```
return next(req);
```

An interceptor should always forward the request unless it intentionally blocks it.

8. Registration

Interceptors are registered globally.

Example

```
provideHttpClient(  
  withInterceptors([  
    authInterceptor  
  ])  
)
```

This registration happens once in

```
app.config.ts
```

From then on,

every HttpClient request passes through the interceptor automatically.

9. Current Responsibilities

Your `authInterceptor` currently performs:

- Reading the JWT

- Cloning the request
- Adding the Authorization header
- Forwarding the request

Nothing else.

10. Future Interceptors

As your project grows,
you'll add more interceptors.

Examples

Authentication Interceptor

↓

Loading Interceptor

↓

Error Interceptor

↓

Logging Interceptor

↓

Backend

Each interceptor should have one responsibility.

11. Typical Interceptor Layout

A well-organized interceptor usually follows this order.

Imports

↓

Interceptor Declaration

↓

Dependency Injection



Request Processing



Forward Request

Best Practices

- ✓ One interceptor should have one responsibility.
 - ✓ Keep interceptors lightweight.
 - ✓ Avoid business logic.
 - ✓ Register interceptors globally.
 - ✓ Use services for shared functionality.
 - ✓ Always forward the request unless intentionally stopping it.
 - ✓ Chain multiple small interceptors instead of creating one large interceptor.
-

Interview Questions

1. What is an Angular Interceptor?
2. Why do we use Interceptors?
3. What is `HttpInterceptorFn`?
4. What are the `req` and `next` parameters?
5. Why do we use `req.clone()`?
6. How is an interceptor registered?
7. Explain how your `authInterceptor` works.
8. Why should authentication logic be placed inside an interceptor?
9. What future interceptors could your project include?
10. How should an interceptor be organized for readability?

Chapter 5: Anatomy of Angular Models and Interfaces

"Models define the shape of your application's data. They allow Components, Services, and the Backend to communicate using well-defined contracts instead of loosely structured objects."

Chapter Overview

In this chapter, we'll learn:

- What Models are
- What Interfaces are
- Why Angular uses Interfaces instead of Classes
- Request Models
- Response Models
- Generic Models
- How Models are used throughout the application
- Best practices

We'll use the models from your `core/models` folder as examples.

Models in Your Project

Your project currently contains:

```
ApiResponse<T>
GenerateRequest
GenerateResponse
JobStatusResponse
LoginRequest
LoginResponse
```

All of these are TypeScript interfaces.

Standard Model Structure

A typical model looks like this.

```
export interface LoginRequest {  
  
    email: string;  
  
    password: string;  
  
}
```

Unlike Components and Services,

models only describe data.

They contain no logic.

1. What is an Interface?

An Interface defines the structure of an object.

Example

```
export interface LoginRequest {  
  
    email: string;  
  
    password: string;  
  
}
```

This means every `LoginRequest` object must contain:

- email
- password

Both must be strings.

2. Why Use Interfaces?

Without interfaces

```
const request = {  
  
    email: "...",  
  
    password: "..."
```

```
};
```

Angular has no guarantee about the object's structure.

With an interface

```
const request: LoginRequest = {  
  email: "...",  
  password: "..."  
};
```

TypeScript verifies the object at compile time.

3. Request Models

Request models represent data sent to the backend.

Examples

```
LoginRequest
```

```
↓
```

```
POST /login
```

```
-----
```

```
GenerateRequest
```

```
↓
```

```
POST /generate
```

They define exactly what the backend expects.

4. Response Models

Response models represent data returned by the backend.

Examples

LoginResponse

↓

JWT

Email

GenerateResponse

↓

JobId

Status

JobStatusResponse

↓

Status

Result

Error

They define exactly what the frontend expects to receive.

5. Generic Models

Your project contains

```
ApiResponse<T>
```

This is a Generic Interface.

```
export interface ApiResponse<T> {  
    success: boolean;  
    message: string;
```

```
data: T;

errors: string[] | null;

}
```

Instead of creating

```
LoginApiResponse

GenerateApiResponse

UserApiResponse
```

Angular uses one reusable wrapper.

6. Understanding Generic Types

Suppose

```
T = LoginResponse
```

The model becomes

```
ApiResponse<LoginResponse>
```

Meaning

```
{

  success,

  message,

  data: LoginResponse,

  errors

}
```

Later

```
T = UserResponse
```

The same interface can be reused.

7. Where Models are Used

Models are shared across the application.

```
Component
↓
Service
↓
HttpClient
↓
Backend
↓
Response
↓
Service
↓
Component
```

Every layer uses the same contract.

8. Login Example

LoginComponent creates

```
const request:  
LoginRequest
```

↓

AuthService sends

```
LoginRequest
```

↓

Backend returns

```
ApiResponse<LoginResponse>
```

↓

Component receives

```
LoginResponse
```

Every object follows its interface.

9. Generate Example

GenerateComponent creates

```
GenerateRequest
```

↓

AiService sends it

↓

Backend returns

```
GenerateResponse
```

↓

Polling returns

```
JobStatusResponse
```

Each request and response has its own model.

10. Why Models Matter

Models provide

- Type Safety
- IntelliSense
- Compile-time validation
- Clear API contracts
- Easier maintenance

Instead of guessing available properties,
developers know exactly what each object contains.

11. Typical Model Layout

A model file usually contains

```
export interface ModelName {  
    property1: type;  
    property2: type;  
}
```

Nothing more.

No constructor.

No methods.

No business logic.

12. Models vs Services vs Components

Each serves a different purpose.

Type	Responsibility
Component	UI
Service	Business Logic
Model	Data Structure

Keeping these responsibilities separate makes applications easier to understand and maintain.

Best Practices

- ✓ Create one interface per model.
 - ✓ Keep models simple.
 - ✓ Do not add business logic to models.
 - ✓ Use descriptive names.
 - ✓ Separate Request and Response models.
 - ✓ Use Generic models when possible.
 - ✓ Keep models inside a dedicated `models` folder.
-

Interview Questions

1. What is an Angular Model?
2. What is a TypeScript Interface?
3. Why do Angular applications use Interfaces?
4. What is the difference between a Request Model and a Response Model?
5. What is a Generic Interface?
6. Explain `ApiResponse<T>`.
7. Why are models important?
8. Where are models used in your application?
9. Why shouldn't models contain business logic?

10. How should models be organized in an Angular project?

Chapter 6: Anatomy of Angular Configuration Files

"Configuration files define how the Angular application starts, how navigation works, and which global services are available. Unlike Components and Services, they don't implement business logic—they configure the framework."

Chapter Overview

In this chapter, we'll learn:

- Purpose of Angular configuration files
- `main.ts`
- `app.config.ts`
- `app.routes.ts`
- `app.component.ts`
- How these files work together
- Best practices

We'll use your current project structure as the reference.

Configuration Files in Your Project

Your application currently contains:

```
main.ts  
  
↓  
  
app.config.ts  
  
↓  
  
app.routes.ts  
  
↓  
  
app.component.ts
```

These four files form the application's startup configuration.

1. main.ts

The application starts here.

```
import { bootstrapApplication }
from '@angular/platform-browser';

import { appConfig }
from './app/app.config';

import { AppComponent }
from './app/app.component';

bootstrapApplication(

  AppComponent,

  appConfig

);
```

Responsibilities:

- Application entry point
- Bootstraps Angular
- Loads the root component
- Loads global configuration

Every Angular application has a single starting point.

2. app.config.ts

This file registers global providers.

Example

```
export const appConfig:

ApplicationConfig = {

  providers: [

    ...

  ]

}
```

```
};
```

Your current providers are:

- Router
- HttpClient
- Authentication Interceptor
- Global Error Listeners

Anything registered here becomes available throughout the application.

3. app.routes.ts

This file defines the application's navigation.

Example

```
export const routes: Routes = [  
  
  ...  
  
];
```

Responsibilities:

- URL mapping
- Route protection
- Parent-child routes
- Redirects

Current routes:

```
/login  
  
/dashboard  
  
/ai/generate
```

The Router uses this file to decide which component should be displayed.

4. app.component.ts

This is the Root Component.

```
@Component({  
  ...  
})  
  
export class AppComponent {  
}
```

Responsibilities:

- Root component
- Hosts the Router
- Entry point for the UI

Your current AppComponent is intentionally minimal.

5. app.component.html

Your HTML contains only

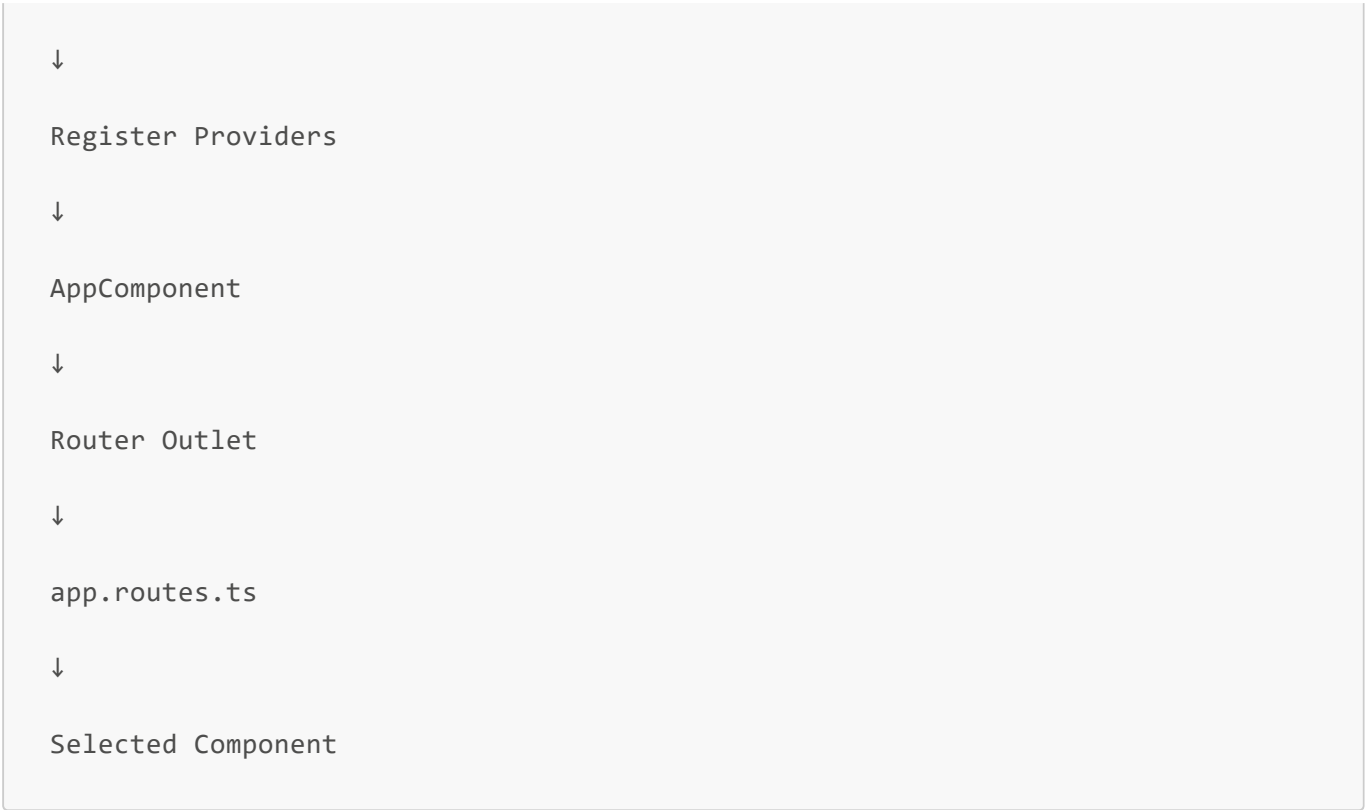
```
<router-outlet>  
  
</router-outlet>
```

The Router replaces this placeholder with the appropriate component based on the current URL.

6. How They Work Together

Application startup follows this sequence.

```
main.ts  
  
↓  
  
bootstrapApplication()  
  
↓  
  
app.config.ts
```

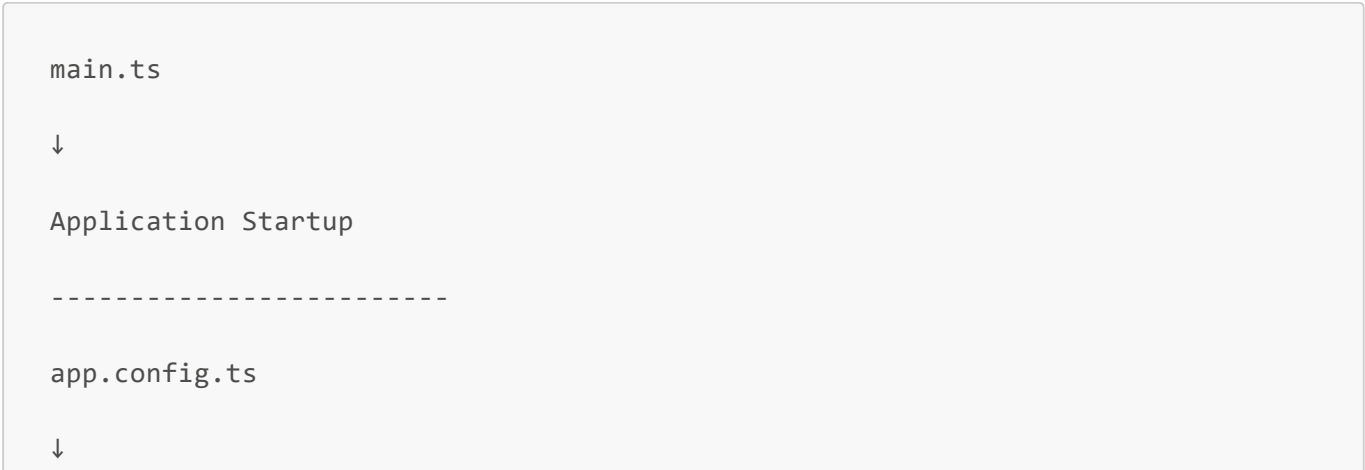


Each file has a distinct responsibility.

7. Responsibilities

File	Responsibility
main.ts	Application entry point
app.config.ts	Global configuration
app.routes.ts	Navigation configuration
AppComponent	Root UI component

8. Typical Organization



Global Services

app.routes.ts

↓

Navigation

AppComponent

↓

Root UI

Keeping startup responsibilities separated makes the application easier to maintain.

Best Practices

- ✓ Keep `main.ts` minimal.
- ✓ Register global providers in `app.config.ts`.
- ✓ Keep routing inside `app.routes.ts`.
- ✓ Keep `AppComponent` lightweight.
- ✓ Avoid placing business logic inside configuration files.
- ✓ Give each configuration file one clear responsibility.

Interview Questions

1. What is the purpose of `main.ts`?
2. What does `bootstrapApplication()` do?
3. What belongs inside `app.config.ts`?
4. What is `ApplicationConfig`?
5. What is the purpose of `app.routes.ts`?
6. Why is `AppComponent` called the Root Component?

7. What is the purpose of `<router-outlet>`?
8. How do the configuration files work together?
9. Why should configuration files not contain business logic?
10. How should Angular startup configuration be organized?

Chapter 7: Angular Project Structure and Folder Organization

"As Angular applications grow, organization becomes just as important as functionality. A well-structured project allows developers to quickly locate files, understand responsibilities, and scale the application without creating confusion."

Chapter Overview

In this chapter, we'll learn:

- Why folder organization matters
- The purpose of each top-level folder
- Feature-based architecture
- Core folder
- Shared folder
- Layouts
- Models
- Services
- Guards
- Interceptors
- How your Smart AI Dashboard is organized
- Best practices

We'll use your current project structure as the reference.

Current Project Structure

```
src
├── app
│   ├── core
│   ├── features
│   ├── layouts
│   ├── shared
│   └── app.component.ts
```

```
├── app.config.ts
├── app.routes.ts
├── main.ts
├── styles.css
└── material-theme.scss
```

Each folder has a single responsibility.

1. src

The **src** folder contains the application's source code.

```
src
↓
Entire Angular Application
```

Everything that Angular compiles lives here.

2. app

The **app** folder contains the actual application.

```
app
↓
Application Code
```

This is where almost all development takes place.

3. core

The **core** folder contains application-wide functionality.

Your project currently contains

```
core
├── guards
├── interceptors
├── models
└── services
```

The Core folder contains things used throughout the application.

Typical contents include:

- Authentication
- API Services
- Models
- Guards
- Interceptors
- Constants
- Utilities

4. features

The **features** folder contains the application's pages and business features.

Your project

```
features
├── auth
├── dashboard
├── ai
├── documents
├── jobs
└── settings
```

Each folder represents one feature of the application.

This is called

Feature-Based Organization.

5. Feature Folder

Each feature contains everything related to that feature.

Example

```
login  
├─ login.component.ts  
├─ login.component.html  
├─ login.component.css  
└─ login.component.spec.ts
```

Keeping files together makes navigation easier.

6. layouts

Layouts define the application's overall page structure.

Your project contains

```
layouts  
├─ auth-layout  
└─ main-layout
```

Examples

```
Toolbar  
  
Sidebar  
  
Navigation  
  
Footer  
  
Router Outlet
```

Layouts are shared across multiple pages.

7. shared

The Shared folder contains reusable resources.

Typical contents include

```
shared
├── components
├── directives
├── pipes
└── material
```

Anything placed here should be reusable by multiple features.

Examples

- Custom Button
- Loading Spinner
- Search Box
- Date Pipe
- Shared Material Module

8. services

Your services live inside

```
core/services
```

Examples

```
AuthService
AiService
TokenService
```

```
AuthStateService
```

All business logic is centralized here.

9. models

Models define data contracts.

```
core/models  
  
↓  
  
Request Models  
  
↓  
  
Response Models  
  
↓  
  
Generic Models
```

Every Component and Service shares these models.

10. guards

Guards protect routes.

```
core/guards  
  
↓  
  
auth.guard.ts  
  
↓  
  
Future:  
  
guest.guard.ts
```

Guards control navigation.

11. interceptors

Interceptors process every HTTP request.

```
core/interceptors
↓
auth.interceptor.ts
↓
Future:
error.interceptor.ts
loading.interceptor.ts
```

They handle cross-cutting concerns.

12. Component Organization

Each component lives in its own folder.

Example

```
generate
├─ generate.component.ts
├─ generate.component.html
├─ generate.component.css
└─ generate.component.spec.ts
```

Everything related to one component stays together.

13. Why Feature-Based Organization?

Imagine organizing by file type.

```
components  
  
services  
  
css  
  
html
```

Finding related files becomes difficult.

Feature-based organization keeps everything together.

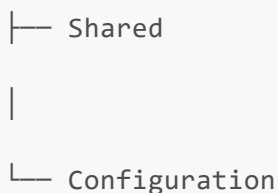
```
Login  
  
↓  
  
TS  
  
HTML  
  
CSS  
  
Tests
```

Much easier to maintain.

14. Current Architecture

Your current project follows this structure.

```
Application  
  
|  
├── Core  
  
|  
├── Features  
  
|  
├── Layouts  
  
|
```



```
graph LR; Root[ ] --- Shared[Shared]; Root --- Config[Configuration];
```

This is the organization used by many enterprise Angular applications.

15. Folder Responsibilities

Folder	Responsibility
core	Global application logic
features	Business features/pages
layouts	Application layouts
shared	Reusable resources
models	Data contracts
services	Business logic
guards	Route protection
interceptors	HTTP pipeline

Best Practices

- ✓ Organize by feature, not by file type.
 - ✓ Keep reusable code inside **shared**.
 - ✓ Keep global services inside **core**.
 - ✓ Give every folder a single responsibility.
 - ✓ Keep related files together.
 - ✓ Avoid deeply nested folder structures.
 - ✓ Follow a consistent structure throughout the application.
-

Interview Questions

1. Why is project organization important?

2. What belongs inside the `core` folder?
3. What belongs inside the `shared` folder?
4. What are Feature folders?
5. Why organize by feature instead of file type?
6. What is the purpose of the `layouts` folder?
7. What belongs inside `models`?
8. What belongs inside `services`?
9. Explain the folder structure of your Smart AI Dashboard.
10. How should a scalable Angular project be organized?

Chapter 8: Angular Coding Conventions and Naming Standards

"Good code is not just code that works—it is code that every developer on the team can quickly read, understand, and maintain. Coding conventions create consistency across the entire project."

Chapter Overview

In this chapter, we'll learn:

- Why coding conventions matter
- File naming conventions
- Class naming conventions
- Folder naming conventions
- Import ordering
- Member ordering
- Access modifiers
- Naming variables and methods
- Code formatting
- Best practices

We'll use your Smart AI Dashboard as the reference project.

1. Why Coding Standards Matter

Imagine joining a project with 2000 files.

If every developer writes code differently,

the project quickly becomes difficult to understand.

Coding standards provide:

- Consistency
- Readability
- Maintainability
- Faster onboarding
- Easier debugging

Enterprise projects strictly follow coding standards.

2. File Naming

Angular follows a predictable naming convention.

Components

```
login.component.ts  
  
dashboard.component.ts  
  
generate.component.ts
```

Services

```
auth.service.ts  
  
ai.service.ts  
  
token.service.ts
```

Guards

```
auth.guard.ts  
  
guest.guard.ts
```

Interceptors

```
auth.interceptor.ts  
error.interceptor.ts
```

Models

```
login-request.ts  
login-response.ts  
api-response.ts
```

Every file clearly indicates its purpose.

3. Class Naming

Class names use **PascalCase**.

Examples

```
LoginComponent  
DashboardComponent  
GenerateComponent  
AuthService  
TokenService  
AuthStateService  
MainLayoutComponent
```

The suffix immediately identifies the file type.

Suffix	Example
Component	LoginComponent
Service	AuthService
Guard	AuthGuard
Interceptor	AuthInterceptor

Suffix	Example
Directive	HighlightDirective
Pipe	CurrencyPipe

4. Folder Naming

Folders use **kebab-case**.

Examples

```
login
dashboard
main-layout
auth-layout
core
shared
features
```

Avoid spaces and PascalCase for folder names.

5. Import Ordering

A consistent import order improves readability.

Recommended order:

```
// Angular
import ...

// Third-party libraries
import ...

// Application Models
import ...
```

```
// Application Services

import ...

// Relative imports

import ...
```

Your current project already follows this style closely.

6. Member Ordering

Keep every component and service organized in the same order.

Components

```
Imports
↓
@Component
↓
Properties
↓
Constructor
↓
Public Methods
↓
Private Methods
```

Services

```
Imports
↓
@Injectable
```

↓

Fields

↓

Constructor

↓

Public Methods

↓

Private Methods

Using the same structure everywhere makes files easier to scan.

7. Access Modifiers

Always specify visibility when appropriate.

```
private authService: AuthService

private router: Router

private http: HttpClient
```

Use:

Modifier	Meaning
private	Internal use only
public	Accessible everywhere
protected	Accessible by subclasses

Most injected services should be **private**.

8. Variable Naming

Use meaningful names.

Good

```
loginForm  
  
errorMessage  
  
loading  
  
currentStatus  
  
resultText  
  
jobId
```

Avoid

```
x  
  
data  
  
obj  
  
temp  
  
value
```

Names should explain their purpose.

9. Method Naming

Methods should describe an action.

Examples

```
login()  
  
logout()  
  
generate()  
  
getStatus()  
  
pollJobStatus()  
  
saveToken()  
  
removeToken()
```

Method names should usually start with a verb.

10. Property Naming

Properties should describe state.

Examples

```
loading  
hidePassword  
errorMessage  
currentStatus
```

State names should describe what the component currently knows.

11. Interface Naming

Use descriptive names.

Examples

```
LoginRequest  
LoginResponse  
GenerateRequest  
GenerateResponse  
JobStatusResponse  
ApiResponse<T>
```

Avoid abbreviations unless universally understood.

12. Keep Responsibilities Small

Each file should have one responsibility.

Examples

AuthService

↓

Authentication

TokenService

↓

Token Storage

AiService

↓

AI Requests

Avoid creating large classes that handle multiple concerns.

13. Code Formatting

A consistent layout improves readability.

Example order

Imports

↓

Decorator

↓

Properties

↓

Constructor

↓

Methods

Use consistent indentation.

Group related code together.

Leave blank lines between logical sections.

Your recent formatting commit follows this style well.

14. Comments

Use comments sparingly.

Good comments explain **why**, not **what**.

Example

```
// Save JWT after successful authentication
```

Avoid comments like

```
// Create variable  
  
const x = ...
```

The code should already make that obvious.

15. Consistency

The most important convention is consistency.

If every component follows the same structure,

developers can immediately locate:

- Properties
- Constructor
- Methods
- Dependencies

without searching through the file.

Your Smart AI Dashboard

Your project already follows many enterprise conventions.

- ✓ Feature-based folders
- ✓ Standalone Components
- ✓ Clear file names
- ✓ PascalCase class names
- ✓ Kebab-case file names
- ✓ Dedicated Services
- ✓ Dedicated Models
- ✓ Thin Components
- ✓ Reusable Core folder

As the project grows, continuing to follow these conventions will keep the codebase easy to maintain.

Best Practices

- ✓ Follow Angular's naming conventions.
 - ✓ Use descriptive names.
 - ✓ Keep one responsibility per file.
 - ✓ Organize members consistently.
 - ✓ Keep formatting consistent.
 - ✓ Prefer readability over clever code.
 - ✓ Write code that another developer can understand quickly.
-

Interview Questions

1. Why are coding conventions important?
2. How are Angular Components typically named?
3. How are Services named?
4. What naming convention is used for folders?
5. How should imports be organized?
6. How should members inside a Component be ordered?

7. Why should variable names be descriptive?
8. What is the purpose of access modifiers?
9. What makes code maintainable?
10. What coding conventions does your Smart AI Dashboard follow?